Variables, Data Types & Git

Part A: Concept Application and Part B: Stretch Problem

Project: Personal Finance Calculator

Part A:

```
(onboarding_env) (base) niranjanhebli@Niranjans-MacBook-Air-76 assignment-2 % python3 main.py
Welcome to the Employee Benefits Portal - Finance Tool

Enter Employee Name: Priya Sharma
Enter Annual Salary: ₹1200000
Enter Tax Bracket (%): 30
Enter Monthly Rent: ₹25000
Enter Savings Goal (%): 20
═══════════════════════════════════════════
EMPLOYEE FINANCIAL SUMMARY
═══════════════════════════════════════════
Employee        : Priya Sharma
Annual Salary   : ₹1,200,000.00
───────────────────────────────────────────
Monthly Breakdown:
Gross Salary    : ₹ 100,000.00
Tax (30.0%)     : ₹ 30,000.00
Net Salary      : ₹ 70,000.00
Rent            : ₹ 25,000.00 (35.7% of net)
Savings (20.0%) : ₹ 14,000.00
Disposable      : ₹ 31,000.00
───────────────────────────────────────────
Annual Projection:
Total Tax       : ₹ 360,000.00
Total Savings   : ₹ 168,000.00
Total Rent      : ₹ 300,000.00
═══════════════════════════════════════════
```

Part B:

```
(onboarding_env) (base) niranjanhebli@Niranjans-MacBook-Air-76 assignment-2 % python3 finance_calculator.py
--- Personal Finance Tool: Comparison Mode ---

Enter Employee Name: Niranjna
Annual Salary (₹): 200000
Tax Bracket (%): 30
Monthly Rent (₹): 2000
Savings Goal (%): 20

Enter Employee Name: Someone
Annual Salary (₹): 2000000
Tax Bracket (%): 30
Monthly Rent (₹): 4000
Savings Goal (%): 40

──────────────────────────────────────────────────────
Metric              | Niranjna        | Someone
──────────────────────────────────────────────────────
Monthly Net         | ₹11,666.67      | ₹1,16,666.67
Rent Paid           | ₹2,000.00       | ₹4,000.00
Rent % of Net       | 17.1            | 3.4
Monthly Savings     | ₹2,333.33       | ₹46,666.67
Health Score        | 100.0           | 100.0
```

Pylint Score:-

```
Report saved to: setup_report.txt
(onboarding_env) (base) niranjanhebli@Niranjans-MacBook-Air-76 assignment-2 % pylint finance_calculator.py
************* Module finance_calculator
finance_calculator.py:12:0: C0303: Trailing whitespace (trailing-whitespace)
finance_calculator.py:15:0: C0303: Trailing whitespace (trailing-whitespace)
finance_calculator.py:19:0: C0303: Trailing whitespace (trailing-whitespace)
finance_calculator.py:25:0: C0303: Trailing whitespace (trailing-whitespace)
finance_calculator.py:39:0: C0303: Trailing whitespace (trailing-whitespace)
finance_calculator.py:43:0: C0303: Trailing whitespace (trailing-whitespace)
finance_calculator.py:47:0: C0303: Trailing whitespace (trailing-whitespace)
finance_calculator.py:99:0: C0304: Final newline missing (missing-final-newline)
finance_calculator.py:37:25: C0321: More than one statement on a single line (multiple-statements)
finance_calculator.py:38:27: C0321: More than one statement on a single line (multiple-statements)
finance_calculator.py:41:27: C0321: More than one statement on a single line (multiple-statements)
finance_calculator.py:42:29: C0321: More than one statement on a single line (multiple-statements)
finance_calculator.py:45:33: C0321: More than one statement on a single line (multiple-statements)
finance_calculator.py:46:34: C0321: More than one statement on a single line (multiple-statements)

------------------------------------------------------------------
Your code has been rated at 7.50/10 (previous run: 8.93/10, -1.43)
```

File Link:

https://github.com/NiranjanHebli/DAY7_PM_Assignment/blob/main/finance_calculator.py

Part C: Interview Ready

Q1: Conceptual — Data Types Deep Dive

Here is the explanation for each line of code. These examples highlight how Python manages logic, memory, and math.

1. print(type(True))
    - Output: <class 'bool'>
    - Explanation: True is a boolean object, representing one of the two logical values in Python.
2. print(isinstance(True, int))
    - Output: True
    - Explanation: In Python, the bool class is a subclass of int. Internally, True is treated as the integer 1.
3. print(True + True + False)
    - Output: 2

    - Explanation: Because booleans are integers, Python calculates this as 1 + 1 + 0
4. print(int(3.99))
    - Output: 3
    - Explanation: Converting a float to an int always truncates the decimal. It does not round to the nearest number; it simply removes the .99.
5. print(bool("False"))
    - Output: True
    - Explanation: Any non-empty string is considered "Truthy." Even though the word is "False," the string itself is not empty, so it returns True.
6. print(bool(""))
    - Output: False
    - Explanation: An empty string contains no data, which Python evaluates as "Falsy."
7. print(0.1 + 0.2 == 0.3)
    - Output: False
    - Explanation: This is due to floating-point precision. Computers store decimals in binary, which can create tiny rounding errors. 0.1 + 0.2 actually equals 0.30000000000000004.
8. print("5" + "3")
    - Output: 53
    - Explanation: Because these are strings (text), the + operator performs concatenation, which glues the characters together.
9. print(5 + 3)
    - Output: 8
    - Explanation: Because these are integers, the + operator performs standard mathematical addition.

Q2: Coding — Build a type analyzer function

File Link:-
https://github.com/NiranjanHebli/DAY7_PM_Assignment/blob/main/type_analyser.py

type_analyser.py > ...

```python
def analyze_value(value):

    # 2. Determine the Truthiness (bool evaluation)
    truthy = bool(value)

    # 3. Handle length calculation safely
    try:
        length = len(value)
    except TypeError:
        # This executes if the value type has no len() method
        length = "N/A"

    return f"Value: {value} | Type: {val_type} | Truthy: {truthy} | Length: {length}"

# --- Testing the Function ---
print(analyze_value(42))        # Integer: No length
print(analyze_value(""))        # Empty string: Falsy, Length 0
print(analyze_value([1, 2, 3])) # List: Truthy, Length 3
print(analyze_value(None))      # NoneType: Falsy, No length
```

PROBLEMS OUTPUT PORTS TERMINAL

TERMINAL

```
(onboarding_env) (base) niranjanhebli@Niranjans-MacBook-Air-76 assignment-2 % python type_analyser.py
Value: 42 | Type: int | Truthy: True | Length: N/A
Value:  | Type: str | Truthy: False | Length: 0
Value: [1, 2, 3] | Type: list | Truthy: True | Length: 3
Value: None | Type: NoneType | Truthy: False | Length: N/A
(onboarding_env) (base) niranjanhebli@Niranjans-MacBook-Air-76 assignment-2 %
```

Q3: Debug — Find and fix 4 bugs

The Bugs Found:-

1. The Comparison Error (Type Mismatch)

The Problem: input() always returns a string. When the code tries to check if age >= 18, it's trying to compare a "word" to a "number," which triggers a TypeError.

The Fix: Wrap the age input in int() to convert it to a number.

2. Variable Swapping

The Problem: In the first print statement, the code says name is {age} years old. This results in something like "25 is 25 years old."

The Fix: Changed the f-string to use the {name} variable first.

3. Math on Strings

The Problem: In the second print statement, {age + 5} would fail because you can't add an integer to a string (if the first bug was missed) or it needs to be part of the logic after conversion.

The Fix: Since we converted age to an int at the top, this math now works perfectly.

4. The Float Formatting

The Problem: {score:.0} is an incomplete format specifier for a float. If you want zero decimal places, it needs to be :.0f. Without the f, Python gets confused about how to handle the precision.

The Fix: Changed the specifier to :.0f.

Corrected Code:-

```
name = input("Name: ")

# BUG 1 FIXED: Converted input to int for comparison and math

age = int(input("Age: "))


if age >= 18:

    status = "Adult"

else:

    status = "Minor"
```

```
# BUG 2 FIXED: Used {name} instead of {age} at the start
print(f"{name} is {age} years old and is a {status}")

# BUG 3 FIXED: This now works because age is an int
print(f"In 5 years: {age + 5}")

score = 85.5
# BUG 4 FIXED: Added 'f' to the format specifier for 0 decimal places
print(f"Score: {score:.0f}")
```

Part D: AI-Augmented Task

Prompt:-

Generate a Python 3.x Type Conversion Matrix in a Markdown table format. The rows should represent the 'Source Type' and the columns should represent the 'Target Type'.

Types to include: int, float, str, bool, list, and tuple.

For each intersection, provide:

The Syntax: e.g., int(x).

The Result: What the output looks like.

Edge Cases: What happens with empty containers ("", []), zero, or None.

Errors: Explicitly state which conversions trigger a ValueError or TypeError (e.g., int("abc") or list(123)).

The AI was highly accurate in its syntax and outcome predictions. It correctly identified the distinction between a ValueError (invalid content, like int("abc")) and a TypeError (invalid type for iteration, like list(123)). The "Non-Iterable Gotchas" section provided practical, idiomatic workarounds that are essential for real-world coding.

Missed Edge Cases & Potential Additions:

- The int to str trap: While the AI mentioned int("1.5") causes a ValueError, it could have noted that int(float("1.5")) is the standard two-step workaround to get an integer from a "float-looking" string.
- Bool to Numeric: The matrix correctly stated int(True) is 1, but it's worth noting that because bool is a subclass of int, we can actually perform math directly on them (e.g., True + True == 2), which is a unique Python behavior.
- Rounding vs. Truncating: The AI correctly used the word "truncate" for int(5.9). However, a developer might expect 6. Adding a mention of round() would provide better context for mathematical conversions.